

TP 1 — Système de Gestion de Bibliothèque Municipale

Alexis PICOT

TP 1 — Système de Gestion de Bibliothèque Municipale

Rappel : L’objectif n’est pas de finir le plus vite possible, mais de comprendre pourquoi la POO est nécessaire. Prenez le temps de réfléchir, de vous tromper, et d’apprendre de vos erreurs.

Bon courage !

Contexte

Vous venez d’être embauché en tant que développeur junior dans une petite entreprise de services numériques. Votre premier client est la bibliothèque municipale de Villeurbanne qui souhaite moderniser son système de gestion.

Le directeur de la bibliothèque, vous explique sa situation :

“Actuellement, tout est géré sur papier et Excel. On a des livres, des adhérents, et on note les emprunts dans un cahier. C’est devenu ingérable avec nos 15 000 ouvrages et 3 000 adhérents. On voudrait un système simple pour gérer tout ça. Rien de compliqué, juste pouvoir enregistrer un emprunt et savoir qui a quoi.”

Le directeur n’est pas très technique. Il vous donne des informations au compte-gouttes et change régulièrement d’avis. Bienvenue dans le monde réel.

Étape 1 — Le besoin initial (version “simple”)

Le directeur vous envoie un premier mail :

“Pour commencer, on voudrait juste pouvoir gérer nos livres. Un livre, c’est simple : un titre, un auteur, un ISBN, et on veut savoir s’il est disponible ou pas. On a aussi besoin de gérer les adhérents : nom, prénom, numéro de carte. Et bien sûr, les emprunts : qui a emprunté quoi et quand.”

Ce que vous devez réaliser

Créez un programme Java qui permet de :

1. **Créer des livres** avec leurs informations de base
2. **Créer des adhérents** avec leurs informations
3. **Enregistrer un emprunt** : un adhérent emprunte un livre
4. **Retourner un livre** : l’adhérent rend le livre
5. **Afficher l’état de la bibliothèque** : liste des livres disponibles, liste des emprunts en cours

Contraintes techniques

- Vous devez pouvoir créer au moins 10 livres et 5 adhérents pour tester
- Un livre ne peut être emprunté que s'il est disponible
- Un emprunt doit enregistrer la date d'emprunt

Questions à vous poser AVANT de coder

- Comment allez-vous représenter un livre ?
- Comment allez-vous représenter un adhérent ?
- Comment allez-vous représenter un emprunt ?
- Comment allez-vous stocker tous ces éléments ?
- Comment allez-vous gérer le lien entre un emprunt, un livre et un adhérent ?

Prenez le temps de réfléchir à votre structure AVANT de coder.

Réponse :

Étape 2 — Première évolution : “Ah oui, j'avais oublié...”

Deux jours plus tard, Le directeur vous rappelle :

“En fait, on ne gère pas que des livres. On a aussi des DVD, des CD audio, et des magazines. Pour les DVD, on a besoin du réalisateur et de la durée. Pour les CD, c'est l'artiste et le nombre de pistes. Pour les magazines, c'est le numéro et la date de parution. Ah, et les durées d'emprunt ne sont pas les mêmes : 3 semaines pour les livres, 1 semaine pour les DVD et CD, et les magazines ne sont pas empruntables, on peut juste les consulter sur place.”

Nouveau cahier des charges

Votre système doit maintenant gérer :

1. **Livres** : titre, auteur, ISBN, disponibilité — emprunt 21 jours
2. **DVD** : titre, réalisateur, durée (en minutes), disponibilité — emprunt 7 jours
3. **CD Audio** : titre, artiste, nombre de pistes, disponibilité — emprunt 7 jours
4. **Magazines** : titre, numéro, date de parution — consultation sur place uniquement (pas d'emprunt)

Ce que vous devez adapter

- Votre système doit pouvoir gérer ces 4 types de documents
- L'emprunt doit calculer automatiquement la date de retour prévue selon le type de document
- Les magazines ne doivent PAS pouvoir être empruntés (le système doit refuser)
- L'affichage doit distinguer les différents types de documents

Questions à vous poser

- Comment allez-vous adapter votre code existant ?
- Allez-vous créer 4 classes différentes ? Une seule classe avec un attribut “type” ?
- Comment allez-vous gérer le fait que les magazines ne sont pas empruntables ?
- Est-ce que votre code de l'étape 1 est facile à modifier ?

Si vous avez l'impression de faire beaucoup de copier-coller, c'est un signe que quelque chose ne va pas.

Réponse :

Étape 3 — Deuxième évolution : “Le conseil municipal a décidé...”

Une semaine plus tard, nouveau coup de fil :

“Le conseil municipal a voté de nouvelles règles. Désormais, les adhérents sont classés en catégories : les enfants (moins de 12 ans), les étudiants, les adultes, et les seniors (plus de 65 ans). Les tarifs d’abonnement sont différents, mais surtout les règles d’emprunt changent : - Les enfants ne peuvent emprunter que 3 documents maximum en même temps - Les étudiants peuvent en emprunter 5 - Les adultes 4 - Les seniors 6

Et les enfants n’ont pas le droit d’emprunter des DVD classés ‘adulte’. Ah, et il faut gérer les retards : si un document est rendu en retard, l’adhérent a une pénalité de 0.20€ par jour de retard. Au bout de 10€ de pénalités, il ne peut plus emprunter.”

Nouvelles règles à implémenter

1. **Catégories d’adhérents** avec des règles différentes :
 - Enfant (< 12 ans) : max 3 emprunts simultanés
 - Étudiant : max 5 emprunts simultanés
 - Adulte : max 4 emprunts simultanés
 - Senior (> 65 ans) : max 6 emprunts simultanés
2. **Restriction sur les DVD** :
 - Ajouter un attribut “classification” aux DVD (Tout public, -12 ans, -16 ans, Adulte)
 - Les enfants ne peuvent emprunter que les DVD “Tout public”
3. **Gestion des pénalités** :
 - Calculer le retard en jours à partir de la date de retour prévue
 - Appliquer 0.20€ par jour de retard
 - Bloquer les emprunts si pénalités $\geq$ 10€
 - Permettre de payer (tout ou partie) des pénalités

Ce que vous devez adapter

- Gérer les différentes catégories d’adhérents
- Vérifier les règles avant chaque emprunt (nombre max, classification, pénalités)
- Calculer et stocker les pénalités

Questions à vous poser

- Combien de `if/else` avez-vous dans votre code maintenant ?
- Si demain on ajoute une catégorie “Famille nombreuse” avec d’autres règles, combien de fichiers devrez-vous modifier ?
- Votre code est-il encore lisible et maintenable ?

C’est le moment de prendre du recul sur votre conception.

Réponse :

Attendus fonctionnels

À la fin du TP, votre programme doit permettre de :

Gestion des documents

- ☐ Créer des livres, DVD, CD audio et magazines
- ☐ Afficher les informations d’un document
- ☐ Lister tous les documents disponibles

- ☐ Lister tous les documents par type
- ☐ Rechercher un document par titre (partiel)

Gestion des adhérents

- ☐ Créer des adhérents de différentes catégories
- ☐ Afficher les informations d'un adhérent (y compris ses emprunts en cours et ses pénalités)
- ☐ Lister tous les adhérents

Gestion des emprunts

- ☐ Enregistrer un emprunt (avec toutes les vérifications nécessaires)
- ☐ Retourner un document (avec calcul des pénalités éventuelles)
- ☐ Afficher les emprunts en cours
- ☐ Afficher les emprunts en retard

Gestion des pénalités

- ☐ Calculer les pénalités d'un adhérent
- ☐ Enregistrer un paiement de pénalités
- ☐ Bloquer les emprunts si pénalités trop élevées

Pièges classiques dans lesquels vous allez probablement tomber

Piège n°1 : La classe "Document" fourre-tout

Vous pourriez être tenté de créer une seule classe `Document` avec un attribut `type` (une `String` ou un `enum`) et tous les attributs possibles (auteur, réalisateur, artiste, durée, nombre de pistes...).

```
Document:
- type: String ("LIVRE", "DVD", "CD", "MAGAZINE")
- titre: String
- auteur: String (null si pas un livre)
- realisateur: String (null si pas un DVD)
- artiste: String (null si pas un CD)
- duree: int (0 si pas un DVD)
- nombrePistes: int (0 si pas un CD)
- numero: int (0 si pas un magazine)
- dateParution: Date (null si pas un magazine)
- ...
```

Pourquoi c'est un piège ? - Beaucoup d'attributs sont null ou à 0 selon le type - Le code est rempli de `if (type.equals("LIVRE"))` partout - Ajouter un nouveau type de document nécessite de modifier cette classe et TOUS les endroits qui l'utilisent - Les bugs sont faciles à introduire (oublier un cas dans un `switch`)

Indice héritage et polymorphisme. Une classe abstraite `Document` avec des sous-classes spécialisées permet d'éviter ces problèmes. Chaque sous-classe ne contient que les attributs qui la concernent, et le polymorphisme évite les `switch/if` sur le type.

Piège n°2 : La classe Adhérent avec des `if/else` pour les règles

Vous pourriez avoir du code comme :

```
if (adherent.getCategorie().equals("ENFANT")) {
    if (adherent.getNombreEmprunts() >= 3) {
        // refuser
```

```

    }
} else if (adherent.getCategorie().equals("ETUDIANT")) {
    if (adherent.getNombreEmprunts() >= 5) {
        // refuser
    }
} else if ...

```

Pourquoi c'est un piège ? - Code difficile à lire et à maintenir - Ajouter une catégorie = modifier tous les if/else - Risque d'oublier des cas - Les règles sont dispersées dans le code au lieu d'être centralisées

Indice héritage ou **composition** avec des **stratégies** peut résoudre ce problème. Chaque type d'adhérent peut avoir sa propre logique de vérification. Les **stratégies** sont un type de design pattern, vous les verrez plus tard dans le cours.

Piège n°3 : Le modèle anémique

Vous pourriez créer des classes qui ne contiennent que des attributs et des getters/setters, puis mettre toute la logique dans une classe "BibliothequeService" ou "GestionnaireEmprunts".

```

class Livre {
    private String titre;
    private String auteur;
    // getters et setters uniquement
}

class BibliothequeService {
    public void emprunter(Livre l, Adherent a) {
        // TOUTE la logique ici
    }
}

```

Pourquoi c'est un piège ? - Les objets ne sont que des "sacs de données" - Toute l'intelligence est dans les services - On n'utilise pas les avantages de la POO - Le code des services devient énorme et difficile à maintenir

Indice encapsulation. Un objet doit savoir faire des choses, pas juste contenir des données. Par exemple, un Document devrait savoir s'il est empruntable, calculer sa date de retour, etc.

Piège n°4 : La gestion des erreurs par des retours de booléens

Vous pourriez écrire :

```

public boolean emprunter(Document doc, Adherent adh) {
    if (!doc.isDisponible()) return false;
    if (adh.getPenalites() >= 10) return false;
    if (adh.getNombreEmprunts() >= adh.getMaxEmprunts()) return false;
    // ...
    return true;
}

```

Pourquoi c'est un piège ? - Impossible de savoir POURQUOI l'emprunt a échoué - L'utilisateur reçoit juste "false" sans explication - Difficile de déboguer - Pas de distinction entre les différents types d'erreurs

Indice exceptions. Elles permettent de gérer les cas d'erreur de manière plus explicite. On peut créer des exceptions spécifiques : `DocumentIndisponibleException`, `PenalitesExcessivesException`, etc.

Piège n°5 : La duplication de code

Vous pourriez avoir le même code copié-collé à plusieurs endroits : - Vérification de disponibilité dans plusieurs méthodes - Calcul de pénalités dupliqué - Logique d’affichage répétée pour chaque type de document

Pourquoi c’est un piège ? - Une modification doit être faite à plusieurs endroits - Risque d’oublier un endroit - Code plus long et plus difficile à lire - Bugs difficiles à corriger

Indice DRY (Don’t Repeat Yourself) **héritage**, **composition** et **méthodes utilitaires** permettent de factoriser le code.

Pour aller plus loin

Niveau 1 — Pour ceux qui terminent l’étape 3

Extension 1.1 : Réservations Le client vous rappelle :

“J’ai oublié de vous dire : on voudrait pouvoir faire des réservations. Si un document n’est pas disponible, un adhérent peut le réserver. Quand le document est retourné, le premier qui l’a réservé est notifié et a 3 jours pour venir le chercher, sinon on passe au suivant.”

Ce que vous devez implémenter : - Système de file d’attente de réservations par document - Notification lors du retour (simulée par un affichage console) - Délai de 3 jours avant passage au suivant - Un adhérent ne peut pas réserver un document qu’il a déjà emprunté

Extension 1.2 : Historique des emprunts

“On voudrait aussi voir l’historique complet des emprunts d’un adhérent, pas juste les emprunts en cours. Et l’historique d’un document aussi : qui l’a emprunté et quand.”

Ce que vous devez implémenter : - Conserver l’historique des emprunts terminés - Afficher l’historique d’un adhérent (avec dates et documents) - Afficher l’historique d’un document (avec dates et emprunteurs)

Niveau 2 — Pour ceux qui vont vite

Extension 2.1 : Recherche avancée

“Les usagers voudraient pouvoir chercher des documents par différents critères : par auteur, par genre, par année de parution... Et combiner les critères !”

Ce que vous devez implémenter : - Ajouter un attribut “genre” aux documents (Roman, SF, Documentaire, Thriller, Comédie, Drame...) - Ajouter une année de parution/sortie - Permettre la recherche par : - Titre (partiel) - Auteur/Réalisateur/Artiste (partiel) - Genre (exact) - Année (exact ou intervalle) - Disponibilité - Permettre de combiner plusieurs critères (ET logique)

Question piège : Comment allez-vous gérer la combinaison de critères sans avoir un nombre explosif de méthodes de recherche ?

C’est le moment d’introduire le pattern **Specification** ou l’utilisation d’**interfaces fonctionnelles** (Predicate) pour créer des filtres composables.

Extension 2.2 : Notifications

“On voudrait envoyer des rappels aux adhérents : quand un emprunt arrive à échéance dans 2 jours, quand il est en retard, quand une réservation est disponible...”

Ce que vous devez implémenter : - Système de notification (simulé par affichage console) - Différents types de notifications (rappel, retard, réservation disponible) - Possibilité pour un adhérent de configurer ses préférences de notification

Question piège : Comment éviter que le code de notification soit dispersé partout dans votre application ?

Indice DP Observer ou d'utiliser des **événements** pour découpler la logique métier de la logique de notification. A priori vous ne devriez pas encore connaître cette manière de faire mais vous pouvez essayer !

Niveau 3 — Pour les très rapides

Extension 3.1 : Multi-sites

“En fait, la bibliothèque a 3 sites dans la ville. Un document appartient à un site. Un adhérent peut emprunter dans n'importe quel site mais doit rendre dans le site d'origine. Si un document réservé est dans un autre site, il faut prévoir un transfert.”

Ce que vous devez implémenter : - Gestion de plusieurs sites - Chaque document appartient à un site - Les adhérents peuvent emprunter dans tous les sites - Retour obligatoire dans le site d'origine (ou gestion des retours inter-sites) - Transfert de documents entre sites pour les réservations - Statistiques par site

Extension 3.2 : Gestion des exemplaires multiples

“Ah oui, j'avais oublié : on peut avoir plusieurs exemplaires du même livre. Par exemple, on a 5 exemplaires de Harry Potter. Quand un adhérent emprunte, il emprunte UN exemplaire, pas LE livre.”

Ce que vous devez implémenter : - Distinction entre “œuvre” (le livre Harry Potter) et “exemplaire” (un exemplaire physique) - Un emprunt concerne un exemplaire, pas une œuvre - Gestion de la disponibilité par exemplaire - État de l'exemplaire (neuf, bon état, usé, à réparer, perdu) - Possibilité de mettre un exemplaire en “indisponible temporairement” (en réparation)

Question piège : Comment adapter votre modèle pour distinguer œuvre et exemplaire sans tout casser ?

Extension 3.3 : Import/Export

“On aimerait pouvoir exporter la liste des documents au format CSV, et aussi pouvoir importer des documents depuis un fichier CSV fourni par nos fournisseurs.”

Ce que vous devez implémenter : - Export CSV de la liste des documents (avec toutes leurs informations) - Import CSV de nouveaux documents - Gestion des erreurs d'import (format invalide, doublons...) - Possibilité d'exporter les emprunts en cours, les retards, etc.

Règles du jeu

Ce qui est INTERDIT

1. **Pas de gros switch/if-else sur les types**
 - Si vous avez plus de 3 branches dans un switch ou if-else qui teste un type, c'est un signe de mauvaise conception
 - Demandez à l'enseignant de valider votre approche
2. **Pas de classes “données pures”**
 - Une classe qui n'a que des attributs et des getters/setters est probablement mal conçue
 - Vos objets doivent avoir des comportements, pas juste des données
3. **Pas de copier-coller**
 - Si vous copiez-collez du code, c'est qu'il y a une opportunité de factorisation
 - Trouvez un moyen de mutualiser le code
4. **Refactorisation obligatoire entre les étapes**
 - Avant de passer à l'étape suivante, prenez le temps de refactoriser votre code

- L'enseignant peut vous demander d'expliquer votre conception
5. **Validation du design avant de continuer**
 - Après l'étape 2, faites valider votre design par l'enseignant
 - Expliquez vos choix et soyez prêt à les remettre en question

Ce qui est **OBLIGATOIRE**

1. **Nommer correctement vos classes et méthodes**
 - Pas de `Classe1`, `methode1`, etc.
 - Les noms doivent refléter le métier
 2. **Commenter les choix de conception**
 - Expliquez pourquoi vous avez fait tel ou tel choix
 - Les commentaires techniques sont moins importants que les commentaires de conception
 3. **Tester manuellement chaque fonctionnalité**
 - Créez un programme de test avec des données réalistes
 - Testez les cas nominaux ET les cas d'erreur
 4. **Gérer les erreurs proprement**
 - Pas de `return null` ou `return false` sans explication
 - Utilisez des exceptions quand c'est approprié
-